Project 2: User Programs

큰 그림 문서 — 특강 후 팀이 함께 채우기

> 담당 키워드를 채운 팀원이 나머지 팀원에게 설명하면서 함께 완성합니다.

시작 시간**: 13:11**
종료 시간**: 23:20**

학습 목표

이 문서를 완성했을 때 우리가 할 수 있어야 하는 것:

- ☐ ELF 로더가 무엇이고 어떤 역할을 하는지 설명할 수 있다
- ☐ 시스템 콜이 왜 필요하고 어떻게 동작하는지 설명할 수 있다
- ☐ 파일 디스크립터가 무엇인지 설명할 수 있다
- ☐ 유저-커널 주소 검증이 왜 필요한지 설명할 수 있다
- ☐ 사용자 프로그램이 실행되는 전체 흐름을 한 문장으로 말할 수 있다

핵심 질문**: Project 1**과 **Project 2**의 결정적 차이는**?**

담당**:** 승진

Project 1에서 우리 코드는 모두 커널에 테스트 코드가 function으로 들어가서 컴파일 됨.

Project 2부터는 테스트 코드가 개별 프로그램으로 컴파일되고, 커널에서 이를 실행하게 됨

이 차이 때문에 반드시 구현해야 하는 것: 파일 시스템과 상호작용하기 위한 시스템 콜을 구현해야함.

1. User Mode vs Kernel Mode

담당**:** 승진

구분	User Mode	Kernel Mode
접근 가능한 메모리	사용자 메모리 영역(KERN_BASE 미만)	전체(사용자 영역 + 커널 영역)
실행 가능한 명령어	커널 기능 요청(int)	사용자 모드 복귀(iret)
잘못된 접근 시	프로세스 즉시 종료(page fault)	사용자 포인터 영역 검증

유저 프로그램이 커널 기능을 사용하려면: 인터럽트 명령어 실행 -> 커널 모드 진입 -> 시스템 콜 호출

이 경계를 넘는 순간: page fault가 발생하고, os에 의해 프로세스가 즉시 중지된다.

[유저 프로그램 **] — User Mode**

|

| ← 이 경계를 넘는 방법: 인터럽트 명령어 실행

|

[커널 **(OS)] — Kernel Mode**

2. Process

담당: 승진

Process란?

정의: user program의 실행 단위

Project 1의 thread와의 차이: project 1의 스레드는 커널 스레드로 vas를 공유하나, project 2에서의 스레드는 user program에 대한 스레드로 독립된 개별 vas(process addr space 라고도 함)을 가짐.

각 프로세스가 자신만의 갖는 것: pas, page table, fd table, pid 등등…

PID (Process Identifier)

PID란: 각 process 식별하기 위한 고유한 id

Pintos에서 PID와 TID의 관계: Pintos는 1 processor 1 thread 인 환경이므로 2개가 동치관계

> 힌트: *Pintos*는 멀티스레드 프로세스를 지원하지 않는다

PEB (Process Environment Block)

PEB란: _____

어떤 정보를 담고 있는가: _____

3. 가상 메모리 레이아웃 (Virtual Memory Layout)

담당: 지운

```
USER_STACK  +-----------------------------------+
            |  ____________________________     |  ← 여기서 시작, 아래로 자란다
            |                 |                 |
            |                 V                 |
            |                                   |
            |                 ^                 |
            |                 |                 |
```

```
            +-----------------------------------+
            |  ______________________________   |  ← BSS 영역
            +-----------------------------------+
            |  ______________________________   |  ← 초기화된 데이터
            +-----------------------------------+
            |  ______________________________   |  ← 코드
0x400000    +-----------------------------------+
            |                                   |
        0   +-----------------------------------+
```

KERN_BASE는: 커널 가상 메모리 주소의 시작점이다. 0 ~ KERN_BASE 미만(사용자 가상 메모리 공간), KERN_BASE 이상(커널 가상 메모리 공간)
단 PintOS에서는 KERN_BASE가 물리 메모리 공간에 1:1 매핑되는 단순한 형태. (계산이 용이해짐)
그럼 KERN_BASE의 끝은 어디일까? KERN_BASE부터 물리 메모리 공간의 끝까지이다.

유저가 접근할 수 없는 주소: KERN_BASE 부터 접근이 불가능하다. 구체적인 값은 0x8004000000

커널이 접근 가능한 메모리: 물리적으로는 모든 메모리 공간에 접근이 가능하다. (KERN_BASE로부터 역계산하면 되니까, 커널 가상 메모리는 전역이기 때문에 모든 프로세스 가상 메모리의 끝은 커널 가상 메모리가 된다.)
하지만 가상 메모리 관점에서 본다면, 커널 메모리와 현재 실행중인 프로세스(RUNNING)의 메모리에 접근이 가능하다. (Page directory base register의 값이 무엇이냐에 따라 메모리 맵 영역이 달라지니까)

Register vs Memory

레지스터를 사용하는 이유: 레지스터가 메모리보다 압도적으로 빠르다.
레지스터 : 1 cycle

L1 캐시 : 4~5 cycle

L2 캐시 : 12 cycle

RAM (메모리): 100~200 cycle

함수 인자를 레지스터로 먼저 넘기는 이유: 1. x86-64 호출 규약이 그렇게 정해져 있다

(규약이 왜 그렇게 정해졌냐면)

2. 스택 메모리보다 레지스터가 압도적으로 빠르기 때문

4. 유저 메모리 접근 — 왜 위험한가?

담당: 지운

커널이 유저가 넘긴 포인터를 그냥 믿으면 안 되는 이유:

유저가 넘길 수 있는 잘못된 포인터 **3**가지**:**

1. NULL 포인터
2. _____
3. _____

처리 방법:
방법 1 (더 단순): _____
방법 2 (더 빠름, Linux 사용): _____

5. ELF & Loader

담당: 규태

ELF (Executable and Linkable Format)란?

정의: 번역: 실행 가능한, 연결 가능한 - 형식

gcc hello.c -o hello 를 실행하면, hello라는 실행 파일이 생긴다.

GNU Compiler Collection///소스코드 파일///출력 파일 이름을 hello로 지정

이 파일이 바로 ELF 파일이다.

리눅스의 표준 실행 파일 형식이며,

pintOS 도 이 형식을 사용_____

ELF 파일이 담고 있는 것:

```
┌──────────────────────┐
│  ELF Header          │ ← 매직 넘버, 아키텍처, 진입점 주소 등
├──────────────────────┤
│  Program Headers     │ ← 어느 영역을 어디에 로드할지의 지시서
├──────────────────────┤
│  .text   (코드)      │
│  .rodata (상수)      │
│  .data   (초기화 변수) │
│  .bss    (미초기화 변수)│
├──────────────────────┤
│  Section Headers     │ ← 링킹용 메타정보
└──────────────────────┘
```

_____

매직 넘버: 코드에 뜻 설명 없이 박혀 있는 하드코딩 숫자

Rodata Read-only data___

Bss block started by symbol 잘못 지은 이름 - 초기값 없는 전역/정적 변수가 들어가는 영역_____

Linux, Pintos가 ELF를 사용하는 이유: _____

Linux가 ELF를 사용하는 이유

1. 역사
2. 기술
   a. 동적 링킹 지원 – 라이브러리를 짊어지지 않고 빌려 쓴다.
   프로그램이 printf 같은 함수를 쓰려면 그 코드가 필요하다. 두 가지 선택 가능.
      i. 동적 링킹: 실행할 때만 빌리겠다고 메모만 적어두고, OS 가 실행 순간에 라이브러리(.so – shared object)를 메모리에 올려 연결.

      두번째. 정적 링킹: printf 코드를 내 실행 파일 안에 통째로 넣음

   b. 여러 아키텍처 호환 – 포장지는 같고 내용물만 다르다
      i. 우편 봉투(ELF)는 어디나 똑같이 생겼다. 안에 들어있는 편지가 한국어든 영어든 봉투 형식은 안 바뀜. 덕분에 OS는 봉투 여는 법 하나만 배우면 됨.

      ELF 파일 맨 앞에는 "이 파일은 x86용입니다" 또는 "ARM용 입니다" 같은 표시가 있다.
      그래서 포장지(ELF형식)는 동일하고,
      안에 든 기계어만 CPU별로 다르다.

   c. 디버깅 정보 내장 – 지도가 같이 들어있다.

      gdb가 바로 쓸 수 있다 (GNU Debugger)

      소스 코드의 줄 번호, 변수 이름, 함수 이름이 함께 들어감

      디버깅할 때, "지금 main.c의 42번째 줄에서 멈추었다. Argc값은 4이다"라고 알려줌

   d. 확장 가능 – 칸막이가 잘 쳐져 있다.
      ELF안은 여러 section 으로 칸이 나뉘어 있다.
      .text – 코드
      .data – 초기화된 변수
      .bss – 초기화 안 된 변수
      .rodata – 상수 문자열
      .symtab – 심볼 표

Pintos가 ELF를 사용하는 이유

Pintos 는 카이스트의 학습용 os 이다.

만약 자기만의 실행 파일 형식을 새로 만들었다면,

학생들이 pintos 전용 컴파일러까지 따로 만들어야 했을 것이다.

그러나 elf 를 채택했기에:

> 학생이 평소 쓰던 gcc로 컴파일한 결과물을 (GNU Compiler Colleciton)
>
> 그대로 pintos 디스크에 올려서
>
> 그대로 실행시킬 수 있다.

Loader가 하는 일

```
ELF 파일
    |
    v
로더가 하는 일:
1. ELF header, Program Header, segment(code, data, BSS)  // 파일에서 무엇을 읽는가?
2. Program header 에 명시된 가상 주소 위치에 올린다 보통 0x400000  // 메모리 어디에
올리는가?
3. ELF 헤더의 e_entry 필드에서 진입점 가상 주소를 읽는다 // 실행 시작점을 어떻게 찾는가?
    |
    v
유저 프로그램 실행
```

Elf header는 “이 파일이 진짜 elf 맞는가, 어떤 cpu용인가, 진입점은 어디인가”를 알려준다

어떤 cpu?

x86-64용 컴파일 결과 (Intel/AMD CPU)

mov rax, 60

syscall

ARM64용 컴파일 결과 (Apple Silicon, 폰)

mov x8, #93

svc #0

Program header 는 “어느 부분을 읽어 어디에 올릴지”

Elf file 안에는 수많은 함수가 있다. Printf, main,...

Entry point address: 0x401050 ← 여기서부터 실행 시작

그 지시에 따라 segment 본문을 디스크에서 읽는다.

Segment? 메모리에 한 덩어리로 올라가는 박스 한 개

왜 덩어리 지는디? - 권한 단위로 메모리에 올리려고

코드 영역: 읽기 + 실행 (쓰기 금지 - 해킹 방지)

데이터 영역: 읽기 + 쓰기 (실행 금지 - 해킹 방지)

상수 영역: 읽기만 (수정 금지)

```
0x400000 근디 이게 뭐임? - 16진수 주소값
   프로그램을 메모리의 가상주소 0x400000 근처에 올리겠다
   숫자로 바꾸면: 0x 가 16진수라는 뜻
               맨 앞 4는 16^5 자리
               뒤 00000 은 다 0
                 그래서 0x400000 = 4 x 16^5 = 4 x 1048576 = 4194304
                 메모리에서 4194304 번 째 바이트 위치. 대략 4MB 지점
```

```
ELF 헤더의 e_entry 필드에서 진입점 가상 주소를 읽는다 // 실행 시작점을 어떻게 찾는가?
    비유)
    책 표지의 "5페이지부터 시작"라는 메모가 붙어있구만 - ELF 헤더의 e_entry
    손가락을 5페이지에 놓자 - intr_frame.rip = e_entry
    "읽기 시작!"명령 - do_iret 명령
    5페이지부터 읽기 시작 - CPU가 그 주소부터 명령어 실행
```

Pintos에서 로더가 구현된 파일: userprog/process.c

6. x86-64 Calling Convention

담당: 규태

함수 호출 시 인자를 전달하는 레지스터 순서: 호출 규약**(calling convention) -** 안 지키면 함수끼리 말안통.

//디스크**(SSD) -** 멀고 느림. 아주 큼.

//메모리(RAM) - 좀 멀지만 빠름. 좀 큼.

순서	레지스터 **- CPU** 안에 있는 아주 작고 아주 빠른 메모지 **16** **- CPU**가 계산을 하려면 값을 어딘가에 잠깐 들고 있어야 함
1번째	%rdi (Register Destination Index, 도착지 표시)_____
2번째	%rsi (Register Source Index, 출발지 표시)_____
3번째	%rdx (Register Data, 데이터 임시 통)_____
4번째	%rcx (Register Counter, 반복 횟수 세는 통)_____
5번째	%r8 (그냥 추가된 것이라 번호만 매김)_____
6번째	%r9 (그냥 추가된 것이라 번호만 매김)_____

인자 개수에 따라 쓰는 메모지가 달라지옵니다

함수	인자 개수	어떤 메모지 쓰나
`halt()`	0개	아무것도 안 씀
`is_even(4)`	1개	`rdi`
`add(3, 5)`	2개	`rdi, rsi`
`write(1, "hi", 2)`	3개	`rdi, rsi, r`
`func(a, b, c, d)`	4개	`rdi, rsi, r`
`func(a, b, c, d, e)`	5개	`rdi, rsi, r`
`func(a, b, c, d, e, f)`	6개	6개 다 씀

반환값을 저장하는 레지스터: %rax_(Register Accumulator, 계산 결과 모으는 통)

//R는 64비트 레지스터라는 뜻

Argument Vector

argc란: argument counting_____

argv란: argument vector_____

argv[0]에 들어가는 것: 실행 파일_____

argv[argc]가 반드시 NULL인 이유: 배열이니까 0부터 시작인. 배열의 끝을 알아야 하니까.

_____

```
1번째 인자 → rdi
2번째 인자 → rsi
3번째 인자 → rdx
...
반환값     → rax
```

`main(int argc, char **argv)`은 인자가 2개이오니:

```
rdi = argc        (인자 개수)
rsi = argv        (인자 배열의 시작 주소)
```

7. System Call

담당: 용

시스템 콜이란?

정의: 유저프로그램이 운영체제(커널)의 기능을 사용하기 위해 커널에 요청하는 공식적인 인터페이스

유저 프로그램이 직접 커널 함수를 호출하지 못하는 이유:

권한 분리 (Privilege Separation). x86_64 기준으로 CPU는 Ring 0(커널) ~ Ring 3(유저)의 권한 레벨을 가진다. 유저 프로세스는 Ring 3에서 동작하며, 커널 메모리·하드웨어 레지스터·특권 명령어(hlt, cli, MMU 조작 등)에 접근할 수 없다. 직접 호출이 가능하다면 권한 모델 자체가 무너진다.

메모리 보호 (Memory Protection). 커널은 자신의 코드/데이터를 가상 주소 공간의 보호된 영역에 둔다. 유저 프로세스가 그 주소로 점프하려 하면 페이지 테이블의 권한 비트(U/S bit)에 의해 페이지 폴트가 발생한다. 즉 주소를 안다고 해도 그냥 부를 수 없다.

신뢰의 경계 (Trust Boundary). 커널은 유저가 넘긴 인자를 절대 신뢰하지 않는다. 시스템 콜이라는 통제된 진입점을 두면 인자 검증, 포인터 유효성 검사, 권한 체크를 일관되게 수행할 수 있다. 직접 호출은 이 검증 단계를 건너뛰게 만든다.

시스템 콜 호출 시 일어나는 일 **(순서대로):**

```
유저: syscall 명령어 실행
    → cpu가 유저모드(ring3) 에서 커널모드 (ring0)로 전환
    → 유저 레지스터/스택 포인터를 커널 스택에 저장 + 커널 스택으로 전환
    → 커널: syscall handler 실행 (systemcall 번호로 디스패치)
    → 반환값을 정해진 레지스터(x86_64에서는 RAX)에 저장 + 유저 컨텍스트 복원
    → 유저로 복귀 (sysret/iret)
```

> ⚠️ *CAUTION: Pintos의 syscall semantics는 POSIX 표준과 다를 수 있다. 반드시 GitBook 기준으로 구현한다.*

Process 관련 syscall

syscall	하는 일	POSIX와 다른 점

halt	pintOS 자체를 즉시 종료	-POSIX에는 이런거 없음 유저 프로세스가 OS 종료는 권한상 불가
exit	현재 프로세스를 종료하고 종료코드를 부모프로세스에 전달 종료시 “프로세스명 : exit(status)” 메시지를 반드시 출력해야 채점됨	프로세스 단위 종료는 같음. 그러나 pintOS는 종료메시지포맷이 명세로 강제되고 기본적으로 1프로세스 1쓰레드라서 _exit() , exit() 구분이 없음
exec	현재 프로세스의 이미지를 새 프로세스로 교체 성공시 절대 돌아오지 않으며 실패시 -1 리턴 , 그리고 cmd_line으로 받는 이유는 shell(bash 등)이 없어서 명령행을 토큰화하고 배열로 넘겨줄 주체가 없음. 그래서 이를 syscall 에 넘겨버림	gitbook에도 evecve 계열과 같다고 함. 똑같이 새 프로세스를 만들지 않음. 인자,환경변수 분리,fd상속 등이 없음 또한 cmd_line 으로 받아서 (*) (|) (>) 처리 불가 공백으로만 단순 분리
fork	현재 프로세스를 그대로 복제해 자식프로세스를 만듬 , 부모는 자식 PID , 자식은 0 을 받는다	posix는 fork()에 인자가 없다. pintos는 자식 스레드 이름을 인자로 받음 copy on write도 없음 그래서 그자리에서 싹다 복제함
wait	자식 프로세스가 종료할 때까지 블록되며, 자식의 exit status를 반환한다. 이미 종료된 자식이라면 즉시 회수한다.	POSIX waitpid와 비슷하지만 옵션 플래그(WNOHANG 등)가 없고, 한 자식은 정확히 한 번만 wait할 수 있다. 두 번째 wait·비-자식 wait 모두 -1.

wait에서 흔히 빠지는 함정

- 자식이 먼저 죽었을 때 : 부모가 wait을 늦게 호출해도 exit status를 받을 수 있어야 한다. 자식의 exit status를 부모가 회수하기 전까지 어딘가에 보관해둬야 한다 (Linux 좀비 프로세스에 해당).
- 부모가 먼저 죽었을 때 : 자식은 고아가 되며 누구도 wait해주지 않는다. 자식 종료 시 자원이 깔끔하게 해제되도록 설계해야 한다.
- 동기화 : fork 직후 자식 프로세스 로딩이 끝났는지 부모가 알 방법이 필요하다. 보통 세마포어로 부모를 블록시켜 자식 로드 성공/실패를 동기화한다.

8. File Descriptor

담당: 용

File Descriptor란?

정의: _____

정수값인 이유: _____

fd 값	의미
0	_____
1	_____
2	_____

File Descriptor Table

누가 가지고 있는가: _____

프로세스가 종료될 때: _____

같은 파일을 두 번 열면: _____

File 관련 syscall

syscall	하는 일
open	_____
close	_____
read	_____
write	_____
seek	_____
tell	_____
create	_____

팀 통합 — 다 채운 후 같이 답하기

> 키워드 담당과 관계없이 팀 전체가 같이 답하는 질문입니다.

Q1. 사용자가 터미널에 **'ls -l'** 을 입력했을 때, 커널 레벨까지 포함한 전체 실행 흐름은?

```
터미널 입력: 'ls -l'
    → _____
    → _____
    → _____
    → _____
    → 화면 출력
```

Q2. 오늘 구현해야 하는 것의 우선순위 순서는?

1순위: _____

2순위: _____

3순위: _____

Q3. 각 구현체가 어느 파일에 들어가야 하는가?

구현할 것	파일
argument passing	_____
syscall handler	_____

유저 메모리 검증	_____
file descriptor table	_____